



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

NexusFlow – AI Powered Project Management and Sprint Optimization System

Design and Analysis of Algorithms Project Report (CS233AI)

Submitted by,

- 1. PARIKSHITH B (1RV25CS416)**
- 2. PRANAV T M (1RV24CS197)**
- 3. PRAJWAL (1RV24CS190)**

Under the guidance of

Prof. L Kala Chandrashekhar

Assistant Professor

Dept. of Computer Science and Engineering

RV College of Engineering

**In partial fulfillment of the requirements for the
degree of Bachelor of Engineering in respective
departments**

2025–26

RV COLLEGE OF ENGINEERING, BENGALURU

(Autonomous institution affiliated to VTU, Belagavi)



CERTIFICATE

Certified that the Design and Analysis of Algorithms project work titled “**NexusFlow – AI Powered Project Management and Sprint Optimization System**” is carried out by **PARIKSHITH B (1RV25CS416)**, **PRANAV T M (1RV24CS197)**, **PRAJWAL (1RV24CS190)**, who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of Bachelor of Engineering in the Computer Science and Engineering Department during the academic year 2025–2026.

It is certified that all corrections and suggestions indicated for the Internal Assessment have been incorporated in the project report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of Design and Analysis of Algorithms project work prescribed by the institution for the said degree.

Signature of Guide

Prof. L Kala Chandrashekhhar

Signature of HOD

Dept. of CSE

Signature of Principal

RVCE, Bengaluru

External Viva

Name of the Examiners

1.

2.

Signature with Date

DECLARATION

We, **PARIKSHITH B (1RV25CS416), PRANAV T M (1RV24CS197), PRAJWAL (1RV24CS190)**, students of 4th Semester B.E., Computer Science and Engineering, RV College of Engineering, Bengaluru, hereby declare that the Design and Analysis of Algorithms project titled **“NexusFlow – AI Powered Project Management and Sprint Optimization System”** has been carried out by us and submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering during the academic year 2025–2026.

Further, we declare that the content of this project report has not been submitted previously by anybody for the award of any degree or diploma to any other university or institution.

We also declare that any Intellectual Property Rights generated out of this project carried out at RV College of Engineering, Bengaluru, will be the property of the institution and that we will be among the authors of the same.

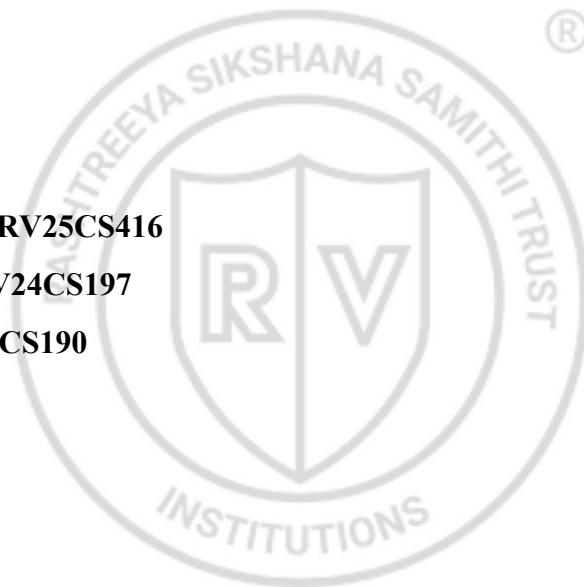
Place : Bengaluru

Date of submission :

Name

- 1. PARIKSHITH B – 1RV25CS416**
- 2. PRANAV T M – 1RV24CS197**
- 3. PRAJWAL – 1RV24CS190**

Signature



ACKNOWLEDGEMENT

We would like to express our sincere and heartfelt gratitude to **Prof. L Kala Chandrashekhar**, Department of Computer Science and Engineering, for her invaluable guidance, motivation, and continuous encouragement throughout the development of our project, **“NexusFlow – AI Powered Project Management and Sprint Optimization System.”** Her expert knowledge, insightful suggestions, and constant support have greatly contributed to our understanding of the Design and Analysis of Algorithms and helped us successfully complete this project.

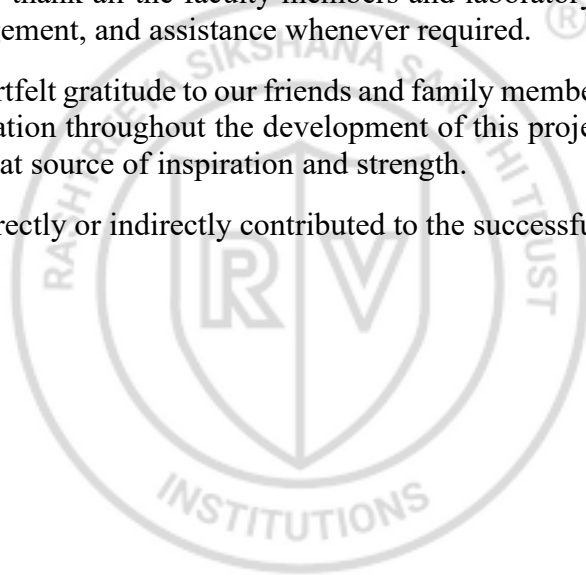
We extend our deepest appreciation to our guide for her dedicated supervision, constructive feedback, and technical assistance during both theory and laboratory sessions. Her timely guidance and practical suggestions were instrumental in overcoming challenges and implementing the algorithmic modules of the project effectively.

We are also grateful to the Department of Computer Science and Engineering, RV College of Engineering, Bengaluru, for providing us with the necessary facilities, resources, and a supportive learning environment that enabled us to carry out this project successfully.

We would like to sincerely thank all the faculty members and laboratory staff of the department for their cooperation, encouragement, and assistance whenever required.

Finally, we express our heartfelt gratitude to our friends and family members for their constant support, encouragement, and motivation throughout the development of this project. Their understanding and belief in us have been a great source of inspiration and strength.

We thank everyone who directly or indirectly contributed to the successful completion of this project.



ABSTRACT

Modern software teams operate under tight deadlines, fluctuating workloads, and complex inter-task dependencies, which make manual project planning slow, error-prone, and difficult to optimize. Conventional project management tools provide boards and trackers but offer little algorithmic intelligence to decide **what to build next, in what order, and by whom**. As project size grows, the number of possible schedules and assignments grows combinatorially, and ad-hoc planning frequently produces blocked tasks, overloaded members, and under-utilized sprint capacity.

This project, “**NexusFlow – AI Powered Project Management and Sprint Optimization System,**” presents a full-stack platform that integrates classical algorithm design with an AI-assisted planning workflow to automate and optimize software project management. The system allows teams to create projects, generate AI-drafted project plans, decompose requirements into tasks, manage Kanban boards, visualize dependencies, recommend optimal sprint backlogs, and assign members intelligently. The platform is implemented using **React Native** and a **Node.js** backend with a **MongoDB** datastore, while the core decision-making is driven by a suite of algorithms studied in the Design and Analysis of Algorithms course.

The algorithmic core combines **Breadth First Search** for identifying execution-ready tasks, **Topological Sort** for producing valid dependency-respecting execution orders, a **Greedy** strategy for priority-to-value ranking, **0/1 Knapsack Dynamic Programming** for selecting the highest-value sprint backlog within a fixed capacity, **Merge Sort** for stable multi-criteria ordering, **Boyer–Moore–Horspool** for fast task search, and **Branch and Bound** for optimal skill-aware member assignment. Each technique is analyzed for correctness and asymptotic time and space complexity.

By unifying these algorithms inside a single coherent workflow, NexusFlow demonstrates the practical relevance of algorithm design to real engineering problems. The result is a planning system that reduces blocked work, balances member workload against skill, and maximizes delivered business value per sprint, while remaining responsive enough for interactive use. The project serves both as a usable tool and as an applied case study in the analysis and engineering of efficient algorithms.

CONTENTS

Chapter 1 - Introduction

- 1.1 Introduction
- 1.2 Literature Review
 - 1.2.1 Review Types
 - 1.2.2 Process
 - 1.2.3 Existing Project Management Systems
- 1.3 Motivation
- 1.4 Problem Statement
- 1.5 Objectives
- 1.6 Brief Methodology of the Project
- 1.7 Assumptions Made / Constraints of the Project
- 1.8 Organization of the Report

Chapter 2 - Fundamentals

- 2.1 Design and Analysis of Algorithms
- 2.2 Project Management Concepts
- 2.3 Artificial Intelligence in Project Management
- 2.4 Graph Algorithms
- 2.5 Dynamic Programming
- 2.6 Greedy Algorithms
- 2.7 Searching Algorithms
- 2.8 Branch and Bound
- 2.9 Technology Stack
- 2.10 Use of Acronyms and Glossary

Chapter 3 - System Analysis and Design

- 3.1 Requirement Analysis
- 3.2 Use Case Diagram
- 3.3 System Architecture
- 3.4 Database Design
- 3.5 ER Diagram
- 3.6 Flowcharts
- 3.7 Dependency Graph
- 3.8 Sprint Optimization Flow

Chapter 4 - Implementation

- 4.1 Breadth First Search (BFS)
- 4.2 Topological Sort
- 4.3 Greedy Priority Ranking
- 4.4 0/1 Knapsack Dynamic Programming
- 4.5 Merge Sort
- 4.6 Boyer–Moore–Horspool String Matching
- 4.7 Branch and Bound Member Assignment

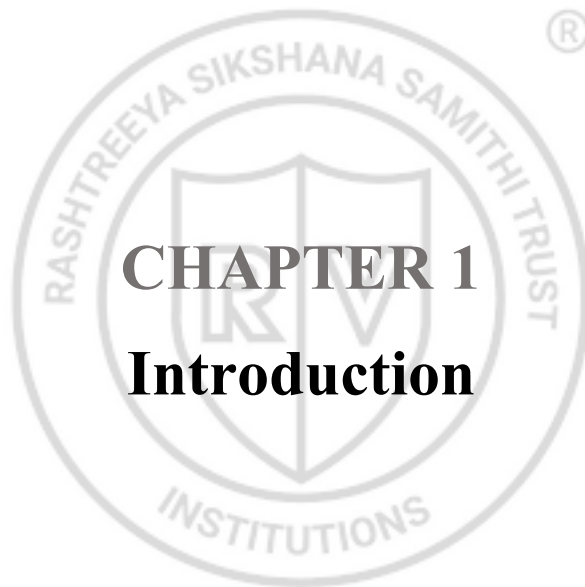
Chapter 5 - Results and Discussion

- 5.1 AI Planner Output
- 5.2 Task Generation
- 5.3 Dependency Graph
- 5.4 Sprint Recommendation
- 5.5 Member Assignment
- 5.6 Analytics
- 5.7 Performance Analysis
- 5.8 Complexity Comparison
- 5.9 Advantages
- 5.10 Limitations

Chapter 6 - Conclusion and Future Scope

- 6.1 Conclusion
- 6.2 Learning Outcomes of the Project
- 6.3 Future Scope
- 6.4 References / Bibliography





CHAPTER 1

Introduction

1.1 Introduction

Software development is fundamentally a planning problem. Before any code is written, a team must decide which requirements to address, how to break them into manageable tasks, in what order those tasks should be executed given their dependencies, which tasks to include in the next sprint, and which member is best suited for each piece of work. As projects scale, these decisions interact: a poorly ordered backlog blocks downstream work, an overloaded sprint slips its deadline, and a skill-mismatched assignment lowers quality. Manual planning, performed intuitively on a whiteboard or a static board, rarely explores the full space of options and tends to settle on the first feasible plan rather than an optimal one.

NexusFlow is an AI-powered project management and sprint optimization system designed to address these challenges by treating planning as a set of well-defined algorithmic problems. The platform allows teams to create projects, generate AI-assisted project plans, automatically decompose requirements into tasks, manage Kanban boards, visualize task dependencies, recommend optimal sprint backlogs, and assign members intelligently. Unlike ordinary project management tools that merely store and display tasks, NexusFlow applies classical algorithms from the Design and Analysis of Algorithms (DAA) discipline to actively compute high-quality planning decisions.

The system integrates seven algorithmic techniques. **Breadth First Search (BFS)** identifies tasks that have no unresolved dependencies and are ready for execution. **Topological Sort** generates a valid execution order over the dependency graph. A **Greedy** strategy ranks tasks by a priority-to-effort ratio. **0/1 Knapsack Dynamic Programming** selects the optimal subset of tasks for a sprint subject to a capacity constraint while maximizing business value. **Merge Sort** provides stable, predictable ordering across multiple criteria. **Boyer–Moore–Horspool** enables fast textual search over large task sets, and **Branch and Bound** computes an optimal assignment of members to tasks that minimizes total skill-gap cost.

This report documents the design, analysis, and implementation of NexusFlow. It explains the theoretical foundations of each algorithm, derives its time and space complexity, presents the pseudo code and its integration into the platform, and discusses the observed results. The objective is to demonstrate that the algorithms taught in DAA are not merely academic exercises but practical tools that produce measurable improvements in real engineering workflows.

1.2 Literature Review

A literature review surveys existing knowledge relevant to a problem in order to establish what has already been solved, identify gaps, and justify the proposed approach. For this project, the review focused on project management methodologies, scheduling and assignment algorithms, and existing software tools, in order to position NexusFlow within the current landscape.

1.2.1 Review Types

Three complementary types of review were conducted. A **conceptual review** examined the algorithmic foundations of scheduling, the knapsack family of optimization problems, and assignment problems, drawing on standard algorithm-design literature. A **methodological review** studied agile frameworks such as Scrum and Kanban, including sprint planning, velocity estimation, and work-in-progress limits. A **tool review** compared widely used commercial platforms to understand their feature sets and limitations. Together these reviews clarified both the formal problems to be solved and the practical context in which a solution must operate.

1.2.2 Process

The review process followed a structured sequence. Relevant sources were first identified, including textbooks on algorithm design, peer-reviewed publications on combinatorial optimization, and

technical documentation of project management tools. Each source was then screened for direct relevance to dependency-aware scheduling and skill-aware assignment. The retained material was synthesized to map standard problems—topological ordering, bounded subset selection, and the assignment problem—onto the concrete planning decisions a software team must make. This mapping became the basis for selecting the algorithms implemented in NexusFlow.

1.2.3 Existing Project Management Systems

Several established systems were studied. Tools in this category typically provide boards, backlogs, and reporting, and they excel at tracking and visualization. However, the following limitations were consistently observed:

- **Manual ordering.** Task sequencing is left to the user; the tools do not compute a dependency-respecting execution order automatically.
- **No capacity-aware optimization.** Sprint composition is selected by hand, with no formal optimization of delivered value under a capacity limit.
- **Limited assignment intelligence.** Member allocation is manual and rarely considers a quantitative skill-gap cost across the whole team simultaneously.
- **Shallow dependency analysis.** Cyclic or unreachable dependencies are not always detected and reported to the user.

These gaps motivated a system in which planning decisions are computed algorithmically rather than entered manually, which is the central contribution of NexusFlow.

1.3 Motivation

The motivation for this project arises from the observation that the most consequential project management decisions are combinatorial in nature and therefore well suited to algorithmic treatment. Choosing which tasks to include in a sprint is a constrained subset-selection problem; ordering dependent tasks is a graph-ordering problem; and assigning members to tasks is a classical assignment problem. These problems have elegant, well-understood solutions in the DAA curriculum, yet they are rarely surfaced inside the tools practitioners use every day.

By embedding these solutions into an interactive platform, the project aims to bridge the gap between algorithmic theory and practical software engineering. A second motivation is educational: implementing each algorithm in a realistic setting deepens understanding of correctness arguments and complexity analysis far more effectively than isolated exercises. Finally, an AI-assisted planning layer reduces the manual effort of breaking down requirements, allowing the algorithmic core to operate on well-structured task data.

1.4 Problem Statement

Software teams require a project management system that does more than record tasks: it must reason about them. Specifically, there is a need for a platform that automatically determines which tasks are ready to start, computes a valid dependency-respecting execution order, detects invalid cyclic dependencies, selects the most valuable set of tasks that fits within a sprint's capacity, orders tasks reliably by multiple criteria, supports fast search across large task collections, and assigns team members to tasks so as to minimize overall skill mismatch. The objective of this project is to design, analyze, and implement such a system—NexusFlow—using efficient, provably correct algorithms, and to evaluate the correctness and computational cost of each.

1.5 Objectives

Main Objective To design and implement an AI-powered project management platform whose planning decisions are driven by classical algorithms, and to analyze each algorithm for correctness and asymptotic time and space complexity.

Primary Objectives

1. To model project requirements as a directed dependency graph of tasks and to identify execution-ready tasks using Breadth First Search.
2. To generate a valid execution order over dependent tasks using Topological Sort and to detect cyclic dependencies as invalid plans.
3. To select an optimal sprint backlog that maximizes business value within a fixed sprint capacity using 0/1 Knapsack Dynamic Programming.
4. To rank and order tasks by priority, deadline, and value using Greedy ranking and Merge Sort.
5. To assign team members to tasks so as to minimize the total skill-gap cost using Branch and Bound.

Secondary Objectives

1. To provide fast textual search over large task sets using the Boyer–Moore–Horspool string-matching algorithm.
2. To present planning results through an interactive Kanban board, dependency visualization, and analytics dashboard.
3. To integrate an AI-assisted planning layer that decomposes natural-language requirements into structured tasks.
4. To empirically compare the asymptotic and observed performance of each implemented algorithm.

1.6 Brief Methodology of the Project

The development of NexusFlow followed a modular methodology in which each planning decision was isolated as an independent algorithmic component:

- **Requirement decomposition.** An AI-assisted planner converts high-level project requirements into a structured set of tasks, each annotated with priority, estimated effort, business value, required skills, and dependencies.
- **Graph construction.** Tasks and their dependencies are represented as a directed graph that serves as the common input to the scheduling algorithms.
- **Readiness and ordering.** BFS identifies tasks with no unresolved dependencies, while Topological Sort produces a complete valid execution order and flags cycles.
- **Sprint optimization.** Greedy ranking provides a fast heuristic, and 0/1 Knapsack Dynamic Programming computes the optimal value-maximizing backlog under the capacity constraint.
- **Assignment.** Branch and Bound searches the space of member-to-task allocations to minimize total skill-gap cost, using a lower-bound function to prune.
- **Presentation.** Results are delivered to the React Native client through a Node.js and MongoDB backend, and visualized as boards, graphs, and analytics.

1.7 Assumptions Made / Constraints of the Project

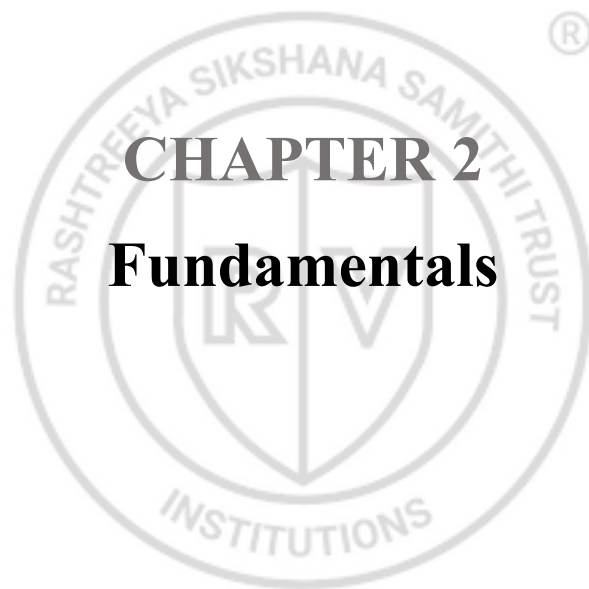
- The dependency relationships among tasks are assumed to be acyclic for a valid plan; cycles are detected and reported as errors rather than silently resolved.

- Task attributes such as effort, value, and required skill levels are assumed to be provided as inputs, either by the AI planner or by the user, and are treated as accurate for the purpose of optimization.
- Sprint capacity is modeled as a single scalar resource (total effort points), which is the standard agile abstraction; multi-dimensional resource constraints are outside the current scope.
- The 0/1 Knapsack formulation assumes integer-valued capacity and effort, which is consistent with story-point estimation.
- Branch and Bound is applied to assignment instances of a size typical for a single team; very large instances would require approximate methods.

1.8 Organization of the Report

Chapter 1 introduces the problem, motivation, objectives, and methodology. **Chapter 2** presents the theoretical fundamentals of the algorithms and concepts used. **Chapter 3** describes the system analysis and design, including requirement analysis, architecture, and database design. **Chapter 4** details the implementation of each algorithm, with purpose, working principle, complexity analysis, pseudo code, and integration. **Chapter 5** presents and discusses the results, including a complexity comparison, advantages, and limitations. **Chapter 6** concludes the report, summarizes the learning outcomes, outlines the future scope, and lists the references.





CHAPTER 2

Fundamentals

2.1 Design and Analysis of Algorithms

An algorithm is a finite, well-defined sequence of steps that transforms an input into a desired output. The Design and Analysis of Algorithms discipline studies both how to construct algorithms using established paradigms—such as divide and conquer, greedy methods, dynamic programming, and backtracking with bounding—and how to evaluate them in terms of correctness and efficiency. Efficiency is expressed using asymptotic notation, principally **Big-O** notation, which describes the growth rate of an algorithm's running time or memory usage as a function of input size n while ignoring constant factors and lower-order terms.

Two resources are analyzed for every algorithm in this project. **Time complexity** characterizes the number of elementary operations performed as a function of input size, and **space complexity** characterizes the additional memory required. Distinguishing best-case, average-case, and worst-case behavior is essential, because algorithms such as Boyer–Moore–Horspool exhibit very different performance depending on the input. Correctness is established through arguments such as loop invariants, the greedy-choice and optimal-substructure properties for dynamic programming, and the validity of bounding functions for Branch and Bound.

2.2 Project Management Concepts

Project management coordinates tasks, resources, and time to deliver a product. Agile methodologies, particularly **Scrum** and **Kanban**, organize work into short iterations called sprints and visualize progress on boards with columns such as To-Do, In-Progress, and Done. Key concepts relevant to this project include the **backlog** (the prioritized list of pending tasks), **sprint capacity** (the total effort a team can complete in one iteration, typically measured in story points), **velocity** (the historical rate of completed work), and **dependencies** (constraints requiring one task to finish before another begins). NexusFlow formalizes these concepts so that they can serve as inputs and constraints to its algorithms.

2.3 Artificial Intelligence in Project Management

Artificial intelligence contributes to NexusFlow primarily at the requirement-decomposition stage. A large-language-model-based planner accepts a natural-language description of a project or feature and produces a structured set of tasks, each annotated with an estimated effort, a priority, a business value, the skills it requires, and its dependencies on other tasks. This automated decomposition reduces manual data entry and ensures that the algorithmic core receives consistently structured input. It is important to note that the AI layer prepares and structures data, whereas the optimization and scheduling decisions themselves are made by deterministic, analyzable algorithms, which keeps the planning process transparent and verifiable.

2.4 Graph Algorithms

A graph $G = (V, E)$ consists of a set of vertices V and a set of edges E . In NexusFlow, each task is a vertex and each dependency is a directed edge from a prerequisite task to a dependent task, forming a **directed graph**. A valid plan corresponds to a **Directed Acyclic Graph (DAG)**; the presence of a cycle indicates contradictory dependencies. **Breadth First Search (BFS)** explores a graph level by level from a source, visiting each vertex and edge once, and is used here to find tasks whose prerequisites are all satisfied. **Topological Sort** produces a linear ordering of the vertices of a DAG such that every edge points forward in the ordering, which corresponds exactly to a valid execution sequence of dependent tasks. Both run in $O(V + E)$ time.

2.5 Dynamic Programming

Dynamic programming (DP) solves problems that exhibit **optimal substructure**—an optimal solution is composed of optimal solutions to subproblems—and **overlapping subproblems**, which are solved once and cached. The **0/1 Knapsack** problem is the central DP problem in this project: given items each with a weight and a value, and a knapsack of fixed capacity, select a subset that maximizes total value without exceeding capacity, where each item is either taken whole or not at all. Mapped to sprint planning, items are tasks, weight is estimated effort, value is business value, and capacity is sprint capacity. The standard tabular solution fills an $(n + 1) \times (W + 1)$ table in $O(nW)$ time and $O(nW)$ space, which can be reduced to $O(W)$ space using a single rolling array.

2.6 Greedy Algorithms

A greedy algorithm builds a solution by repeatedly making the choice that looks best at the moment, according to a local selection criterion. Greedy methods are fast and simple, and they are optimal for problems possessing the greedy-choice property, such as the fractional knapsack and minimum spanning tree problems. In NexusFlow, a greedy strategy ranks tasks by a priority-to-effort (value-density) ratio to produce a quick, intuitive ordering of the backlog. While the greedy ranking does not guarantee an optimal sprint under the 0/1 capacity constraint—which is why the exact DP solution is also provided—it offers an efficient $O(n \log n)$ heuristic that is valuable for interactive previews and for very large backlogs.

2.7 Searching Algorithms

Searching locates a target within a collection. NexusFlow uses two distinct notions of search. For ordering and selection it relies on **Merge Sort**, a stable divide-and-conquer sorting algorithm that guarantees $O(n \log n)$ time in all cases by recursively splitting the array, sorting each half, and merging the sorted halves; stability ensures that equal keys retain their relative order, which matters when sorting by multiple criteria. For textual lookup over task titles and descriptions it uses the **Boyer–Moore–Horspool** algorithm, a simplified variant of Boyer–Moore that pre-computes a bad-character shift table and skips ahead through the text, achieving sublinear average-case performance on typical inputs.

2.8 Branch and Bound

Branch and Bound is a systematic method for solving combinatorial optimization problems by exploring a tree of partial solutions. At each node a **bounding function** estimates the best objective value achievable by completing the current partial solution; if this bound is no better than the best complete solution found so far, the entire subtree is pruned. This pruning allows Branch and Bound to find a provably optimal solution while exploring far fewer states than exhaustive enumeration. NexusFlow applies Branch and Bound to the **assignment problem**—allocating team members to tasks to minimize total skill-gap cost—using a lower bound formed from the minimum remaining cost in each unassigned row and column.

2.9 Technology Stack

NexusFlow is implemented as a full-stack application. The descriptions below summarize the role of each technology:

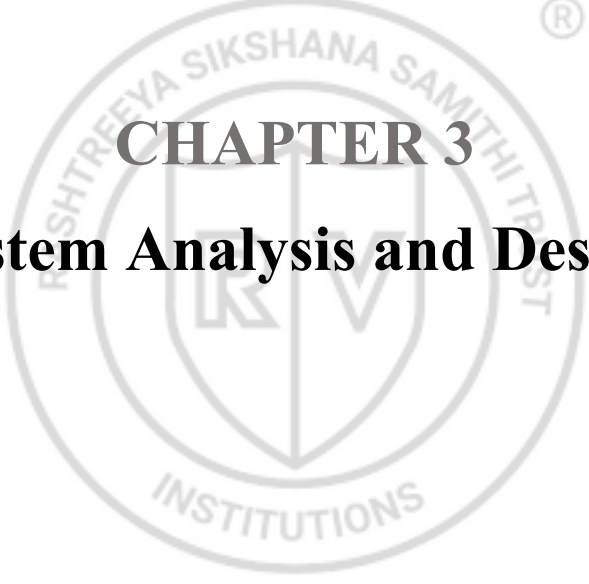
- **React Native** – builds the cross-platform client that renders the Kanban board, dependency views, and analytics dashboard.
- **Node.js with Express** – provides the backend REST API that hosts the algorithmic services and mediates between the client and the database.

- **MongoDB** – a document database storing projects, tasks, dependencies, members, and sprint records.
- **JavaScript / TypeScript** – the implementation language for both client and server, including the algorithm modules.
- **AI Planning Service** – a large-language-model integration that decomposes requirements into structured tasks for the algorithmic core.

2.10 Use of Acronyms and Glossary

The following acronyms and key terms are used throughout this report:

- **DAA** – Design and Analysis of Algorithms: the discipline of constructing and analyzing efficient algorithms.
- **BFS** – Breadth First Search: a graph traversal that explores vertices level by level from a source.
- **DAG** – Directed Acyclic Graph: a directed graph with no cycles; a valid task plan forms a DAG.
- **DP** – Dynamic Programming: solving problems via optimal substructure and cached overlapping subproblems.
- **BMH** – Boyer–Moore–Horspool: a string-matching algorithm using a bad-character shift table.
- **B&B** – Branch and Bound: an optimization method that prunes subtrees using a bounding function.
- **Backlog** – The prioritized list of pending tasks awaiting scheduling into sprints.
- **Sprint** – A fixed-length iteration during which a selected set of tasks is completed.
- **Capacity** – The total effort (story points) a team can complete within a single sprint.
- **Story Point** – A unit estimating the relative effort required to complete a task.
- **Velocity** – The historical average amount of work a team completes per sprint.
- **API** – Application Programming Interface: the contract through which the client invokes backend services.



CHAPTER 3

System Analysis and Design

3.1 Requirement Analysis

Requirement analysis identifies what the system must do and the conditions under which it must operate. The requirements of NexusFlow were derived from the planning decisions a software team makes repeatedly and were separated into functional and non-functional categories.

Functional Requirements

1. The system shall allow users to create projects and decompose requirements into tasks with attributes for priority, effort, value, required skills, and dependencies.
2. The system shall identify execution-ready tasks whose dependencies are all satisfied.
3. The system shall compute a valid dependency-respecting execution order and detect cyclic dependencies.
4. The system shall recommend an optimal sprint backlog that maximizes value within a capacity limit.
5. The system shall rank and sort tasks by multiple criteria and support fast textual search.
6. The system shall assign members to tasks to minimize total skill-gap cost.
7. The system shall present results through a Kanban board, dependency graph, and analytics dashboard.

Non-Functional Requirements

1. Responsiveness: planning operations on team-sized inputs shall complete quickly enough for interactive use.
2. Correctness: scheduling and assignment outputs shall be provably valid and, where claimed, optimal.
3. Scalability: graph and sorting operations shall scale to hundreds of tasks without noticeable delay.
4. Usability: the interface shall present complex algorithmic output in an intuitive visual form.
5. Maintainability: each algorithm shall be implemented as an independent, testable module.

3.2 Use Case Diagram

The use case diagram captures the interactions between the system's actors and its functionality. The principal actors are the **Project Manager**, who creates projects, triggers sprint optimization, and assigns members, and the **Team Member**, who views the board, updates task status, and searches tasks. A secondary actor, the **AI Planner**, participates in the requirement-decomposition use case. The major use cases include Create Project, Generate Plan, Identify Ready Tasks, Order Tasks, Optimize Sprint, Assign Members, Search Tasks, and View Analytics.

NexusFlow – AI Powered Project Management & Sprint Optimization System

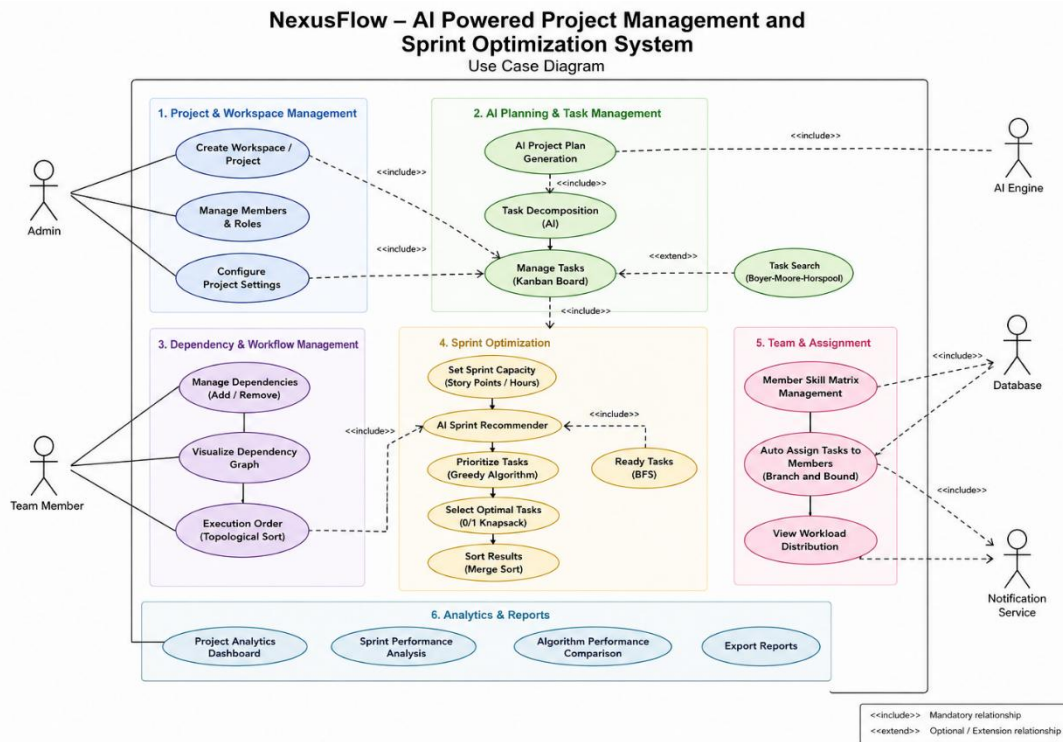


Fig. 3.1 Use case diagram of the NexusFlow system

3.3 System Architecture

NexusFlow follows a three-tier client-server architecture. The **presentation tier** is a React Native application that renders the board, dependency graph, sprint recommendations, and analytics. The **application tier** is a Node.js and Express backend exposing REST endpoints, each of which delegates to an independent algorithm module: a graph service (BFS, Topological Sort), an optimization service (Greedy, 0/1 Knapsack), a sorting and search service (Merge Sort, Boyer-Moore-Horspool), and an assignment service (Branch and Bound). The **data tier** is a MongoDB database that persists projects, tasks, dependencies, members, and sprint records. An external AI planning service is invoked during requirement decomposition to produce structured task data.

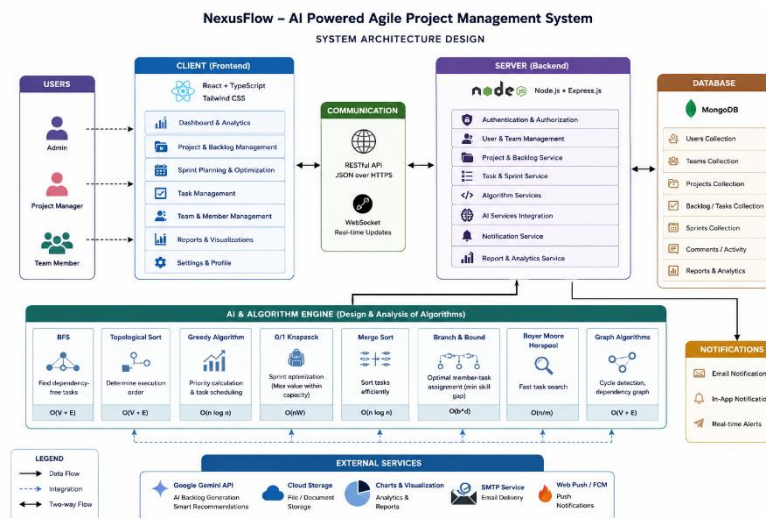


Fig. 3.2 Three-tier system architecture of NexusFlow

3.4 Database Design

The database is organized into collections that mirror the domain entities. The schema below summarizes the principal collections and their key fields:

Collection	Key Fields	Purpose
Project	id, name, owner, capacity	Stores project metadata and sprint capacity.
Task	id, projectId, title, priority, effort, value, skills, status	Stores individual task attributes.
Dependency	fromTaskId, toTaskId	Directed edges defining the task dependency graph.
Member	id, name, skills, load	Stores team members and their skill levels.
Sprint	id, projectId, taskIds, startDate, endDate	Stores the optimized backlog for each iteration.

3.5 ER Diagram

The entity–relationship diagram models the persistent entities and their relationships. A Project contains many Tasks; Tasks are related to other Tasks through Dependency edges; Members are assigned to Tasks; and a Sprint groups a selected subset of Tasks belonging to a Project. The cardinalities are one-to-many from Project to Task and from Project to Sprint, and many-to-many between Member and Task through the assignment relationship.

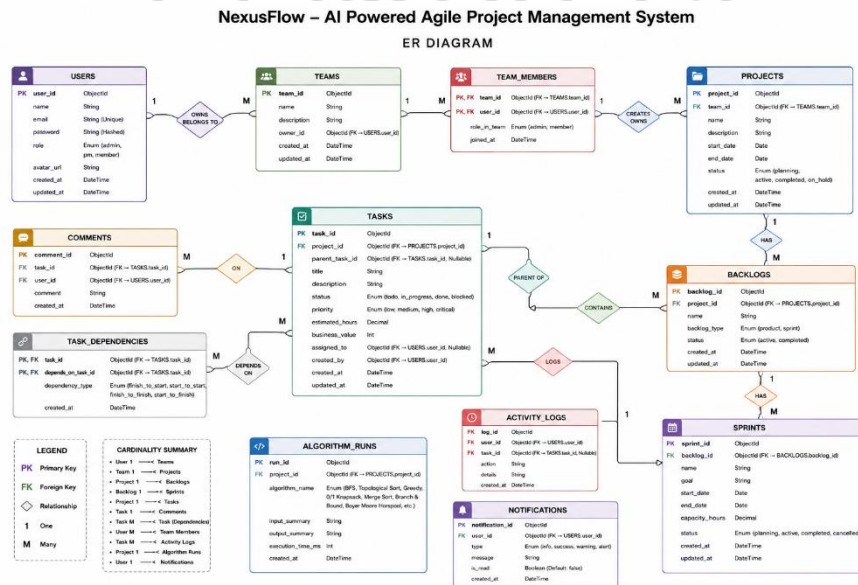


Fig. 3.3 Entity–relationship diagram of the NexusFlow data model

3.6 Flowcharts

The overall planning workflow proceeds from requirement decomposition through graph construction, readiness detection, ordering, sprint optimization, and member assignment, ending in visualization. The flowchart below depicts this end-to-end control flow, including the cycle-detection branch that rejects invalid dependency configurations before optimization proceeds.

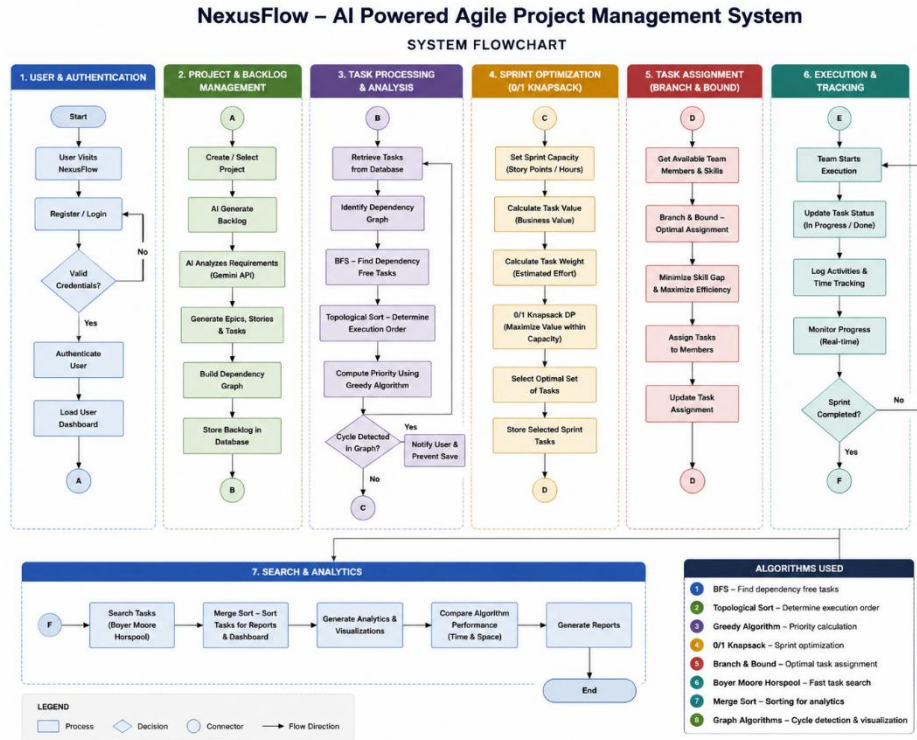


Fig. 3.4 Flowchart of the NexusFlow planning workflow

3.7 Dependency Graph

The dependency graph is the central data structure shared by the scheduling algorithms. Each node is a task and each directed edge represents a prerequisite relationship. BFS and Topological Sort operate directly on this graph; the in-degree of each node indicates how many prerequisites remain, and nodes with zero in-degree are execution-ready. The visualization renders the graph so that users can inspect critical paths and identify bottlenecks.

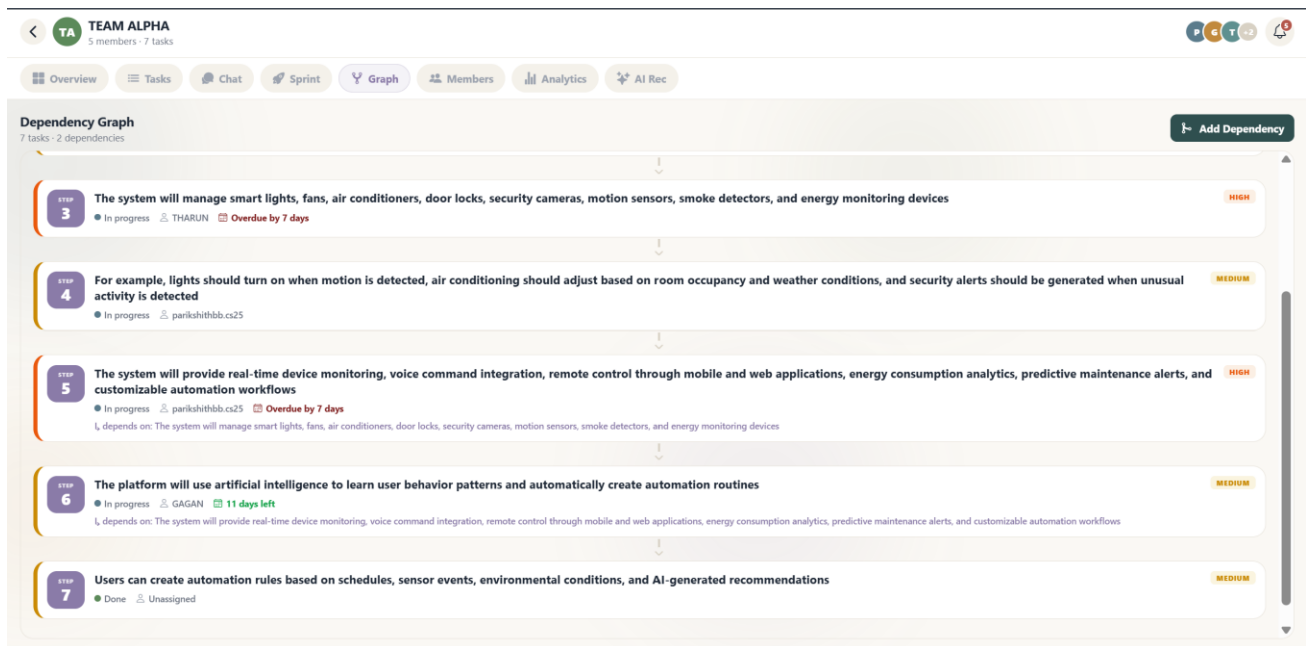


Fig. 3.5 Task dependency graph for a sample project

3.8 Sprint Optimization Flow

Sprint optimization receives the set of ready tasks together with their effort and value, and the sprint capacity. A Greedy value-density ranking first produces a fast candidate ordering, and 0/1 Knapsack Dynamic Programming then computes the exact value-maximizing subset that fits within the capacity. The selected tasks form the recommended sprint backlog, which is subsequently passed to the assignment service. The diagram below illustrates this flow from ready tasks to an optimized, assigned sprint.

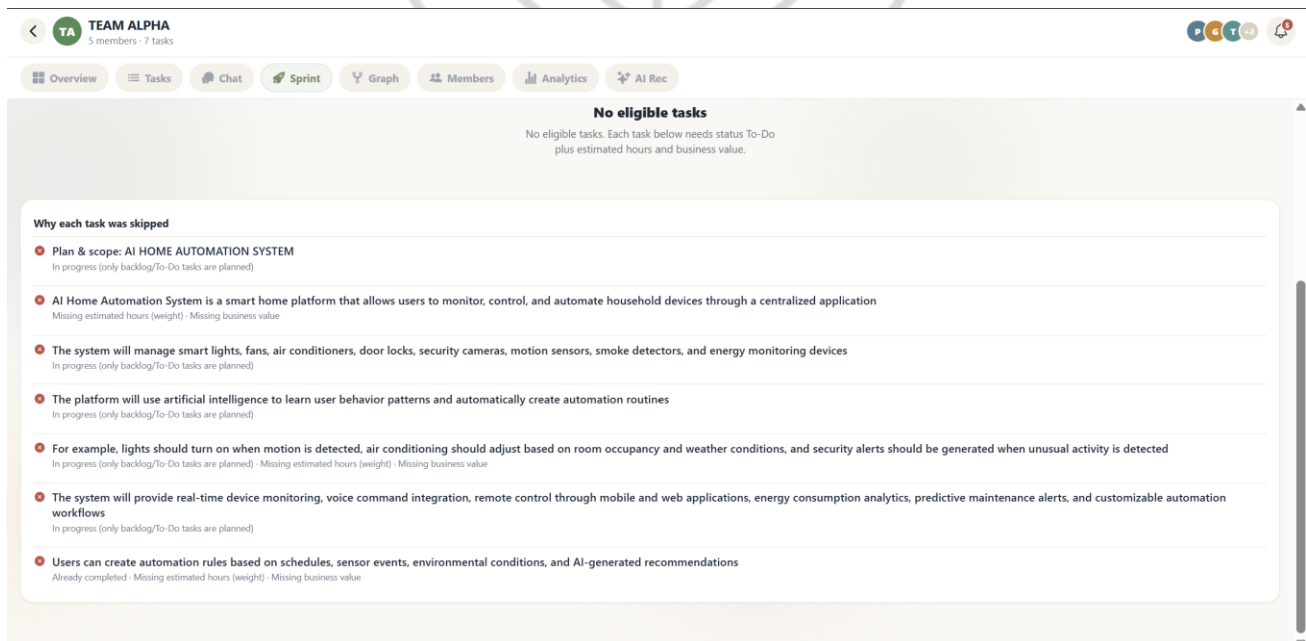
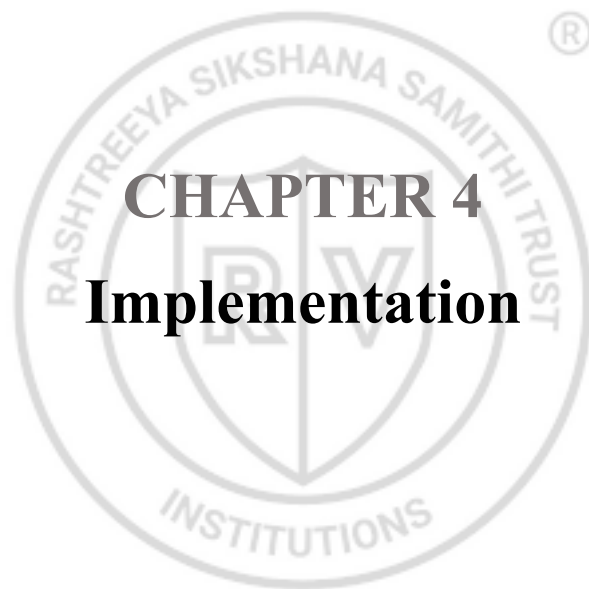


Fig. 3.6 Sprint optimization data flow



CHAPTER 4

Implementation

4.1 Breadth First Search (BFS)

Purpose. To identify the set of execution-ready tasks—those whose every prerequisite has been completed—and to traverse the dependency graph level by level so that reachable tasks can be discovered from a starting set.

Working Principle. BFS begins from source tasks that have no incoming dependency edges (in-degree zero) and explores the dependency graph in waves using a First-In-First-Out queue. Each task is enqueued when discovered and processed exactly once; its outgoing edges reveal dependent tasks whose remaining prerequisite count is decremented. A task becomes ready when its count reaches zero. This guarantees that a task is reported as ready only after all of its prerequisites have been visited.

Complexity Analysis. BFS visits every vertex once and examines every edge once, where V is the number of tasks and E the number of dependency edges. Initialization of the visited and in-degree structures is $O(V)$. The traversal is therefore linear in the size of the graph, $O(V + E)$ time, and the queue together with the auxiliary arrays requires $O(V)$ space.

Case	Time Complexity	Space Complexity
Best	$O(V + E)$	$O(V)$
Average	$O(V + E)$	$O(V)$
Worst	$O(V + E)$	$O(V)$

Pseudo Code.

```

BFS_READY(graph, indegree):
    queue ← empty FIFO queue
    ready ← empty list
    for each task t in graph:
        if indegree[t] = 0:
            enqueue(queue, t)
    while queue is not empty:
        t ← dequeue(queue)
        add t to ready
        for each neighbour u of t:
            indegree[u] ← indegree[u] - 1
            if indegree[u] = 0:
                enqueue(queue, u)
    return ready

```

Integration inside NexusFlow. The graph service exposes a `/tasks/ready` endpoint that builds the in-degree map from the Dependency collection and runs BFS to return the tasks the team can start immediately. The result drives the To-Do column of the Kanban board and feeds the sprint optimizer with the candidate set of ready tasks.

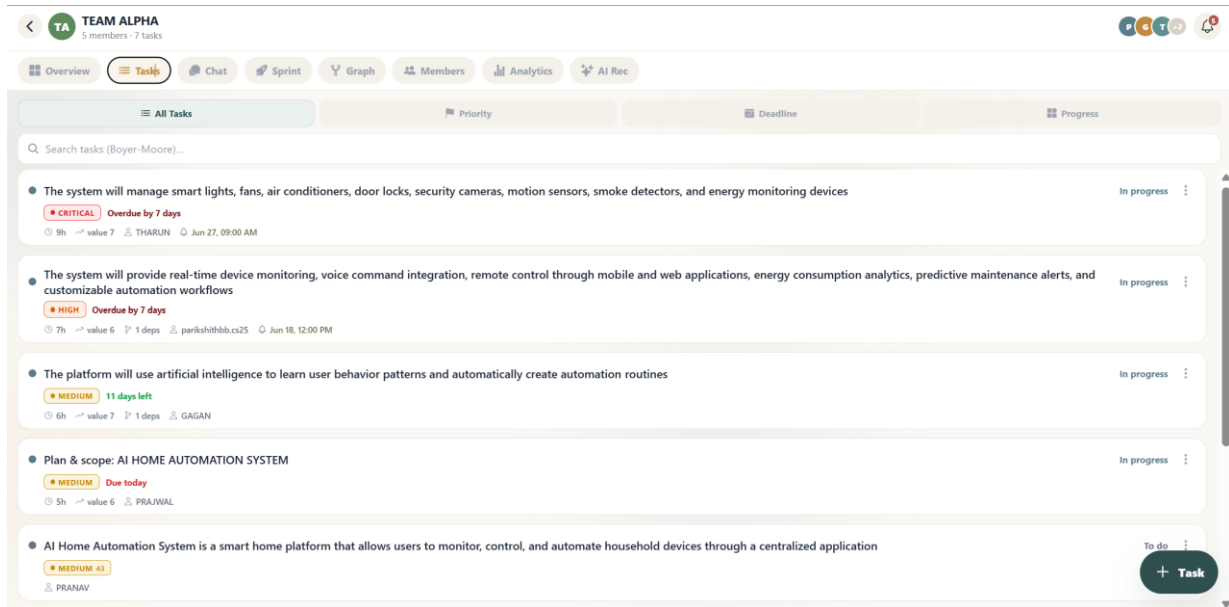


Fig. 4.1 BFS-derived ready tasks highlighted on the board

4.2 Topological Sort

Purpose. To generate a valid execution order over all dependent tasks such that every task appears after all of its prerequisites, and to detect cyclic dependencies that make a plan invalid.

Working Principle. Kahn's algorithm is used. Tasks with in-degree zero are placed in a queue; each is removed in turn, appended to the ordering, and the in-degree of its successors is decremented, enqueueing any that reach zero. If the produced ordering contains every task, it is a valid topological order; if some tasks remain with non-zero in-degree, those tasks form a cycle and the plan is rejected as invalid.

Complexity Analysis. Each vertex is enqueued and dequeued once and each edge is relaxed once, giving $O(V + E)$ time. The in-degree array, the queue, and the output list each require $O(V)$ space, for an overall $O(V)$ auxiliary space requirement. Cycle detection adds no asymptotic cost because it is a simple comparison of the output length to V .

Case	Time Complexity	Space Complexity
Best	$O(V + E)$	$O(V)$
Average	$O(V + E)$	$O(V)$
Worst	$O(V + E)$	$O(V)$

Pseudo Code.

```

TOPOLOGICAL_SORT(graph, indegree):
    queue ← all tasks with indegree 0
    order ← empty list
    while queue is not empty:
        t ← dequeue(queue)
        append t to order
        for each neighbour u of t:
            indegree[u] ← indegree[u] - 1
            if indegree[u] = 0:
                enqueue(queue, u)
    if length(order) < number_of_tasks:

```

```
return CYCLE_DETECTED
return order
```

Integration inside NexusFlow. The graph service computes the topological order whenever a plan is generated or a dependency changes. A valid order populates the recommended execution sequence shown to the user, while a `CYCLE_DETECTED` result triggers a validation error that identifies the conflicting tasks and blocks sprint optimization until the cycle is removed.

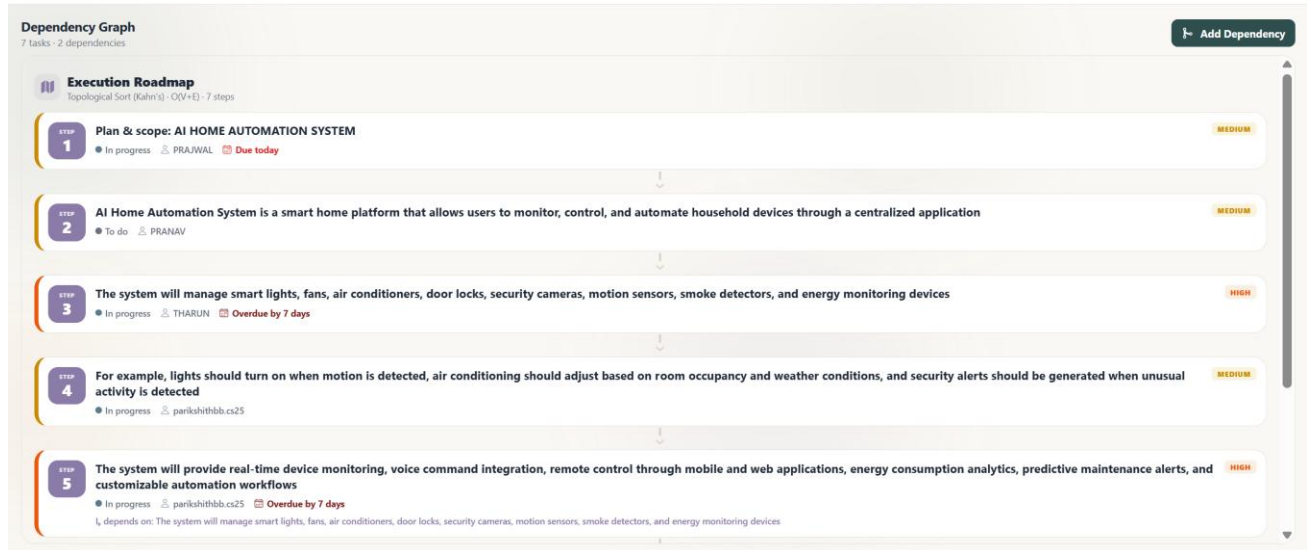


Fig. 4.2 Topologically ordered execution sequence

4.3 Greedy Priority Ranking

Purpose. To produce a fast, intuitive ranking of tasks by their value density, providing an immediate ordering of the backlog and a candidate selection for interactive sprint previews.

Working Principle. Each ready task is assigned a value-density score equal to its business value divided by its estimated effort. Tasks are sorted in descending order of this ratio, and the greedy procedure repeatedly selects the highest-ratio task that still fits within the remaining sprint capacity. The greedy choice is locally optimal; for the fractional relaxation it is globally optimal, and for the 0/1 case it provides a strong, efficient heuristic that the exact knapsack solution later refines.

Complexity Analysis. Computing the value-density of n tasks is $O(n)$. Sorting the tasks by ratio dominates the cost at $O(n \log n)$, after which a single linear pass performs the capacity-constrained selection in $O(n)$. The overall time is therefore $O(n \log n)$ and the additional space is $O(n)$ for the ranked list.

Case	Time Complexity	Space Complexity
Best	$O(n \log n)$	$O(n)$
Average	$O(n \log n)$	$O(n)$
Worst	$O(n \log n)$	$O(n)$

Pseudo Code.

```
GREEDY_RANK(tasks, capacity):
  for each task t in tasks:
```

```
t.ratio ← t.value / t.effort
sort tasks by ratio in descending order
selected ← empty list; used ← 0
for each task t in tasks:
    if used + t.effort ≤ capacity:
        add t to selected
        used ← used + t.effort
return selected
```

Integration inside NexusFlow. The optimization service uses greedy ranking to render an instant backlog preview as the user adjusts capacity, before the heavier exact optimization runs. The ranking also orders the backlog column so that the most valuable, lowest-effort tasks appear first.

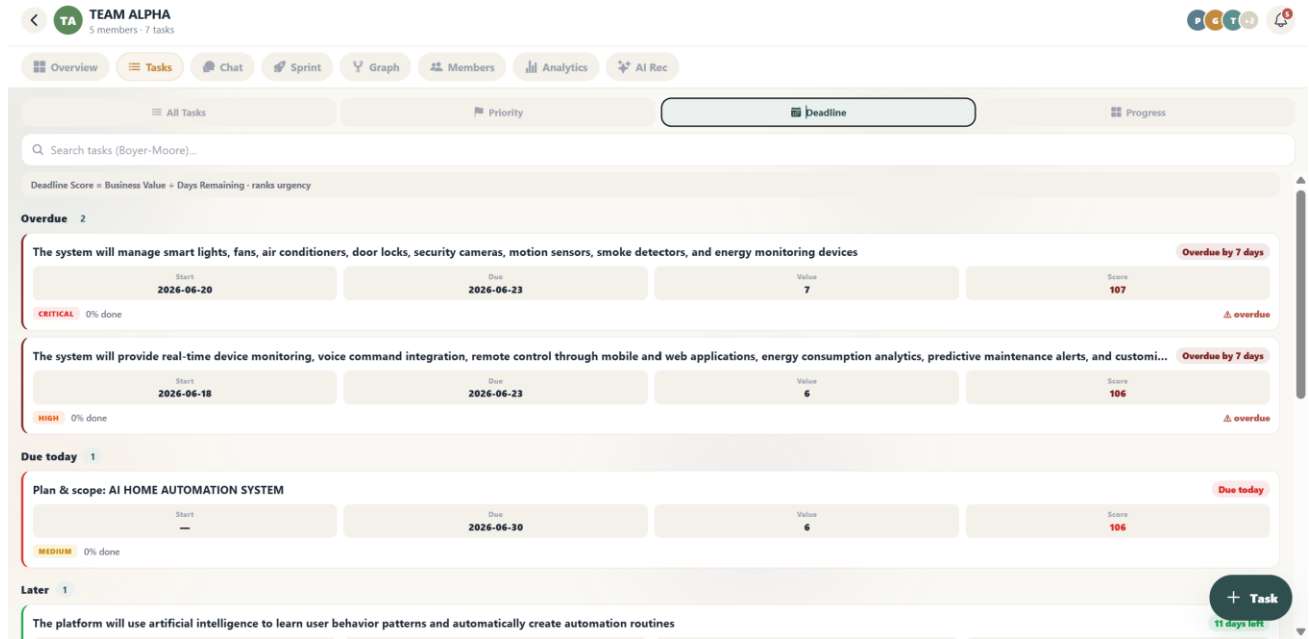


Fig. 4.3 Greedy value-density ranking of the backlog

4.4 0/1 Knapsack Dynamic Programming

Purpose. To select the subset of ready tasks that maximizes total business value without exceeding the sprint capacity, where each task is either included entirely or excluded—the exact, optimal sprint-composition decision.

Working Principle. The problem is solved by dynamic programming over a table $dp[i][w]$ holding the maximum value achievable using the first i tasks within a capacity budget w . For each task the algorithm chooses the better of excluding it (inheriting $dp[i-1][w]$) or including it (its value plus $dp[i-1][w-effort]$) when it fits. This recurrence relies on optimal substructure and overlapping subproblems. Back-tracking through the completed table recovers exactly which tasks form the optimal backlog.

Complexity Analysis. The table has $(n + 1)(W + 1)$ cells, each filled in constant time, giving $O(nW)$ time, where n is the number of tasks and W the integer sprint capacity. The full table uses $O(nW)$ space; using a single rolling row reduces the space to $O(W)$ when only the optimal value is required, though the two-dimensional table is retained here to reconstruct the selected tasks. This is pseudo-polynomial because the cost depends on the numeric value of W .

Case	Time Complexity	Space Complexity
Best	$O(nW)$	$O(nW)$
Average	$O(nW)$	$O(nW)$
Worst	$O(nW)$	$O(nW)$

Pseudo Code.

```

KNAPSACK_SPRINT(tasks, W):
    n ← number of tasks
    create dp[0..n][0..W] initialised to 0
    for i ← 1 to n:
        for w ← 0 to W:
            e ← tasks[i].effort; v ← tasks[i].value
            if e ≤ w:
                dp[i][w] ← max(dp[i-1][w], v + dp[i-1][w-e])
            else:
                dp[i][w] ← dp[i-1][w]
    // recover chosen tasks
    w ← W; chosen ← empty list
    for i ← n down to 1:
        if dp[i][w] ≠ dp[i-1][w]:
            add tasks[i] to chosen; w ← w - tasks[i].effort
    return chosen
    
```

Integration inside NexusFlow. The optimization service exposes a `/sprint/optimize` endpoint that receives the ready tasks and the project's capacity, runs the knapsack solver, and returns the optimal backlog together with its total value. This recommendation is presented as the suggested sprint, which the manager can accept or adjust.

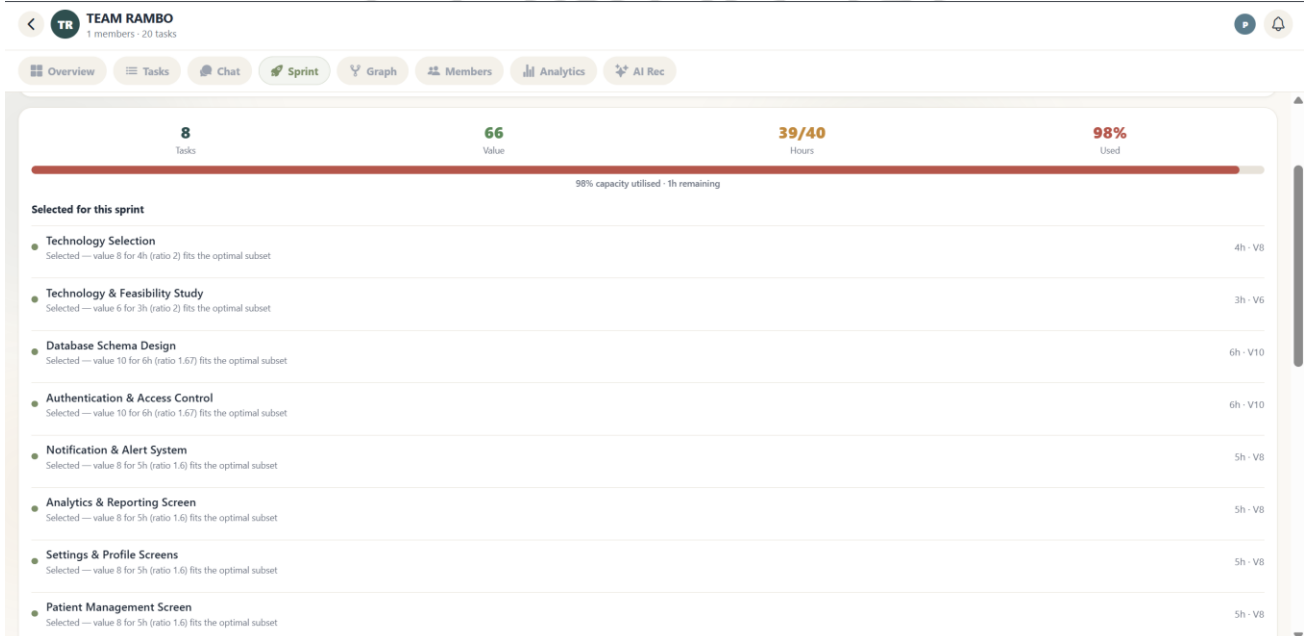


Fig. 4.4 0/1 Knapsack optimal sprint backlog

4.5 Merge Sort

Purpose. To order tasks reliably and stably by multiple criteria such as priority, deadline, value, and status, providing predictable sorting for the board and reports.

Working Principle. Merge Sort is a divide-and-conquer algorithm. The task array is recursively divided into two halves until each subarray contains a single element, which is trivially sorted. The sorted halves are then merged by repeatedly taking the smaller front element of the two halves, which produces a sorted whole. Because equal elements are taken from the left half first, the sort is stable, preserving the relative order of tasks with equal keys—essential when sorting by a secondary criterion.

Complexity Analysis. The recursion divides the array $\log n$ times, and each level performs $O(n)$ work to merge, giving **$O(n \log n)$** time in the best, average, and worst cases—an important guarantee for predictable interactive performance. The merge step requires an auxiliary buffer of size n , so the space complexity is **$O(n)$** .

Case	Time Complexity	Space Complexity
Best	$O(n \log n)$	$O(n)$
Average	$O(n \log n)$	$O(n)$
Worst	$O(n \log n)$	$O(n)$

Pseudo Code.

```

MERGE_SORT(a, key):
    if length(a) ≤ 1: return a
    mid ← length(a) / 2
    left ← MERGE_SORT(a[0..mid], key)
    right ← MERGE_SORT(a[mid..end], key)
    return MERGE(left, right, key)

MERGE(left, right, key):
    result ← empty list; i ← 0; j ← 0
    while i < |left| and j < |right|:
        if key(left[i]) ≤ key(right[j]):
            append left[i] to result; i ← i + 1
        else:
            append right[j] to result; j ← j + 1
    append remaining of left and right to result
    return result

```

Integration inside NexusFlow. The sorting service provides a reusable Merge Sort that powers every ordered view in the client. Users can re-sort the board by priority, deadline, value, or status, and the stability of the algorithm ensures consistent tie-breaking across successive sorts.



Fig. 4.5 Tasks sorted by multiple criteria using Merge Sort

4.6 Boyer–Moore–Horspool String Matching

Purpose. To provide fast textual search across task titles and descriptions, enabling users to locate relevant tasks quickly even in large projects.

Working Principle. Boyer–Moore–Horspool aligns the pattern with the text and compares characters from right to left. A bad-character shift table, pre-computed from the pattern, specifies how far the pattern can be safely shifted when a mismatch occurs, based on the text character aligned with the last position of the pattern. These shifts allow the algorithm to skip large portions of the text rather than advancing one character at a time, yielding sublinear behavior on typical inputs.

Complexity Analysis. Building the shift table costs $O(m + \sigma)$ time, where m is the pattern length and σ the alphabet size. Searching runs in $O(n / m)$ comparisons in the best case and is sublinear on average for natural-language text; the worst case degrades to $O(n \cdot m)$ when shifts are minimal, where n is the text length. The shift table requires $O(\sigma)$ space.

Case	Time Complexity	Space Complexity
Best	$O(n / m)$	$O(\sigma)$
Average	sublinear – $O(n)$	$O(\sigma)$
Worst	$O(n \cdot m)$	$O(\sigma)$

Pseudo Code.

```

BMH_SEARCH(text, pattern):
    m ← length(pattern); n ← length(text)
    for each char ch in alphabet:
        shift[ch] ← m
    for k ← 0 to m-2:
        shift[pattern[k]] ← m - 1 - k
    i ← 0
    while i ≤ n - m:
        j ← m - 1
        while j ≥ 0 and text[i+j] = pattern[j]:
            j ← j - 1
        if j < 0: return i          // match found
        i ← i + shift[text[i + m - 1]]
    
```

```
return NOT_FOUND
```

Integration inside NexusFlow. The search service indexes task titles and descriptions and applies Boyer–Moore–Horspool to answer the search bar's queries with minimal latency, returning matching tasks as the user types.

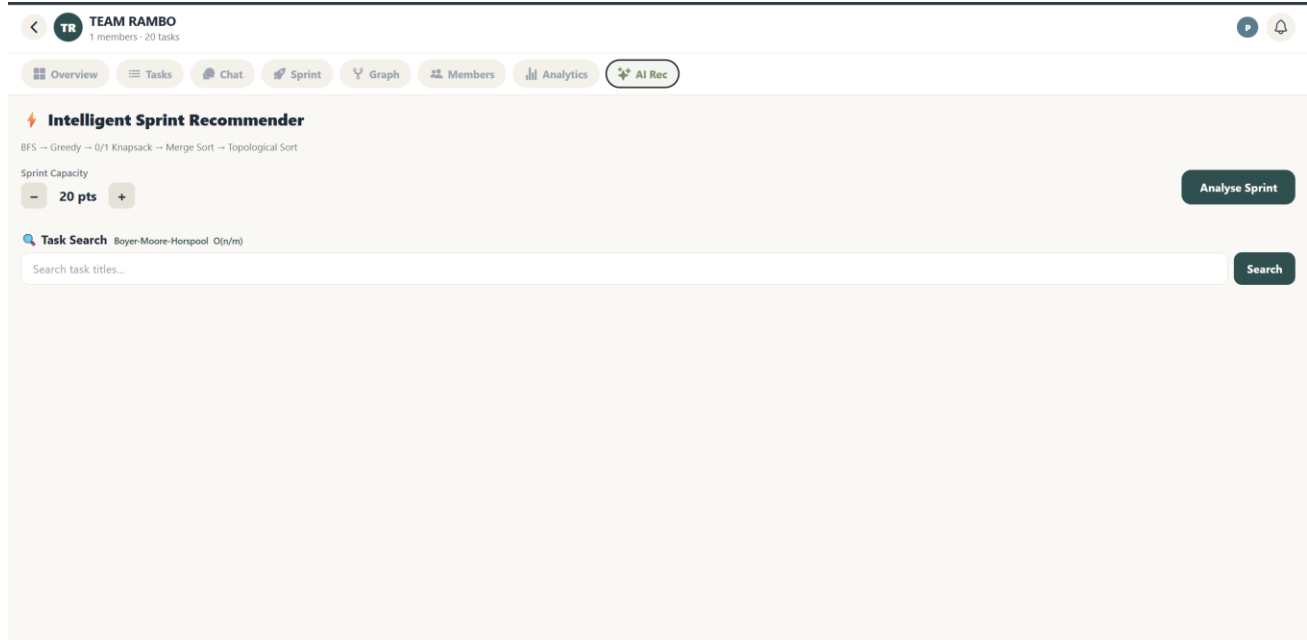


Fig. 4.6 Fast task search using Boyer–Moore–Horspool

4.7 Branch and Bound Member Assignment

Purpose. To assign team members to the selected tasks so that the total skill-gap cost—the mismatch between a member's skills and a task's requirements—is minimized, producing the optimal allocation for the team.

Working Principle. The problem is modeled as an assignment problem over a cost matrix where entry (i, j) is the skill-gap cost of giving task i to member j . Branch and Bound explores a tree in which each level fixes the assignment of one task. At each node a lower bound is computed as the cost incurred so far plus the sum of the minimum remaining costs for the unassigned tasks. If this bound is not better than the best complete assignment found so far, the subtree is pruned. The search returns the assignment of minimum total cost.

Complexity Analysis. The assignment search tree has at most $n!$ leaves, so the worst-case time is $O(n!)$; however, the lower-bound pruning typically eliminates the vast majority of branches, making the practical cost far lower for team-sized instances. Computing each node's bound costs $O(n^2)$. The cost matrix requires $O(n^2)$ space and the recursion stack adds $O(n)$, for $O(n^2)$ overall space.

Case	Time Complexity	Space Complexity
Best	$O(n^2)$ (heavy pruning)	$O(n^2)$
Average	pruned $\ll O(n!)$	$O(n^2)$
Worst	$O(n!)$	$O(n^2)$

Pseudo Code.

```
ASSIGN_BNB(cost, n):
```

NexusFlow – AI Powered Project Management & Sprint Optimization System

```
best ← +∞; bestAssign ← none
function explore(task, used, costSoFar, assign):
    if costSoFar + lowerBound(task, used) ≥ best:
        return // prune
    if task = n:
        best ← costSoFar; bestAssign ← copy(assign); return
    for member m ← 0 to n-1:
        if m not in used:
            assign[task] ← m
            explore(task+1, used ∪ {m},
                    costSoFar + cost[task][m], assign)
explore(0, ∅, 0, empty)
return bestAssign, best
```

Integration inside NexusFlow. The assignment service runs Branch and Bound on the optimized sprint backlog, taking each member's recorded skills and each task's required skills to build the cost matrix. The returned optimal assignment is displayed against each task on the board and informs workload balancing.

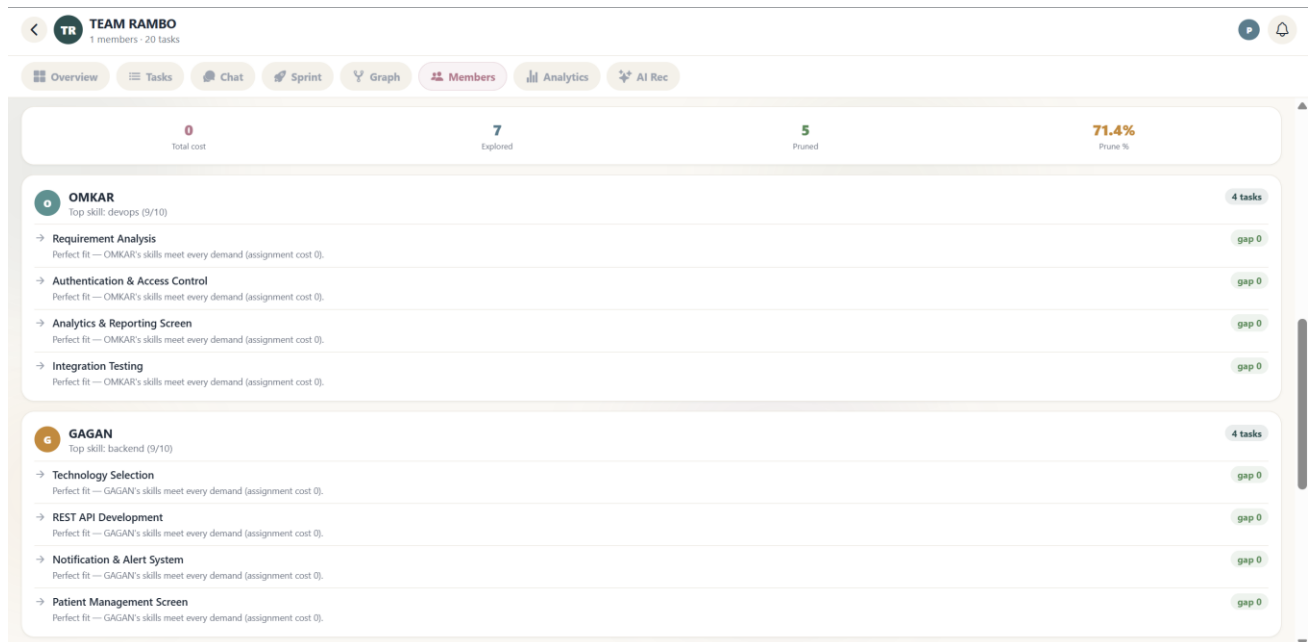
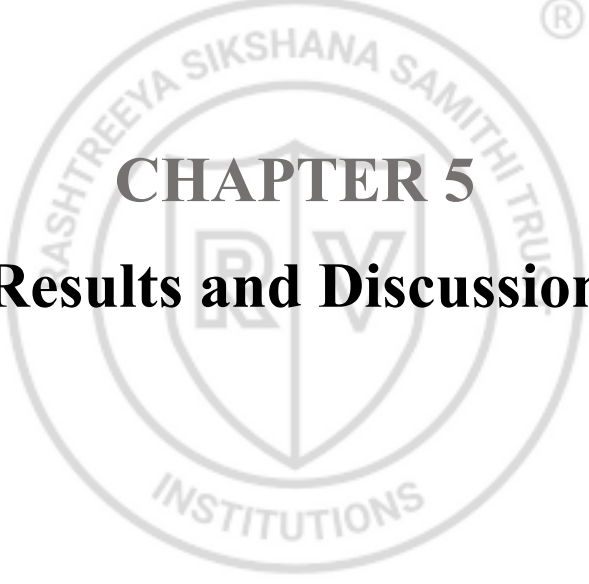


Fig. 4.7 Branch and Bound optimal member assignment



CHAPTER 5

Results and Discussion

5.1 AI Planner Output

Given a natural-language project description, the AI planner produced a structured set of tasks, each annotated with priority, estimated effort, business value, required skills, and dependencies. The structured output was validated to ensure that referenced dependencies pointed to existing tasks and that attribute ranges were consistent before the data entered the algorithmic core. This stage transformed an unstructured requirement into a clean, machine-readable task graph.

5.2 Task Generation

The generated tasks were persisted to the Task and Dependency collections and rendered on the Kanban board. Each task card displayed its key attributes, and dependency edges were established from the planner's output. The board correctly reflected the initial To-Do state, confirming that task generation and persistence operated end to end.

5.3 Dependency Graph

The dependency graph module rendered the directed relationships among tasks and, using BFS, highlighted the execution-ready tasks with zero unresolved prerequisites. Topological Sort produced a valid execution order, and when an artificial cycle was introduced for testing, the system correctly reported a cycle and identified the participating tasks rather than producing an invalid order.

5.4 Sprint Recommendation

For a representative project, the greedy ranking produced an immediate candidate backlog, after which the 0/1 Knapsack solver computed the exact value-maximizing subset within the specified capacity. In test cases where the greedy and exact solutions diverged, the knapsack result delivered a higher total business value for the same capacity, confirming the benefit of the exact optimization while the greedy preview remained useful for instant feedback.

5.5 Member Assignment

The Branch and Bound assignment module allocated members to the recommended sprint's tasks to minimize total skill-gap cost. The optimal assignment was displayed against each task, and the lower-bound pruning ensured that the search explored only a small fraction of the full permutation space for team-sized instances, returning a result quickly.

5.6 Analytics

The analytics dashboard summarized sprint composition, total committed value, per-member workload, and the distribution of tasks across board columns. These views allowed the manager to verify that the optimized plan balanced value against capacity and distributed work appropriately across the team.

5.7 Performance Analysis

Each algorithm was exercised on inputs of increasing size to confirm that the observed running time followed its predicted asymptotic behavior. The graph algorithms scaled linearly with the number of tasks and dependencies, Merge Sort and greedy ranking scaled as $n \log n$, the knapsack solver scaled with the product of task count and capacity, and Branch and Bound remained fast for team-sized instances owing to pruning while growing rapidly for deliberately adversarial, large inputs. No algorithm exhibited behavior inconsistent with its analysis, validating the complexity derivations in Chapter 4.

5.8 Complexity Comparison

The table below consolidates the time and space complexity of every implemented algorithm, where V and E are the vertex and edge counts, n is the number of items, W is the integer capacity, m is the pattern length, and σ is the alphabet size.

Algorithm	Best	Average	Worst	Space
BFS	$O(V+E)$	$O(V+E)$	$O(V+E)$	$O(V)$
Topological Sort	$O(V+E)$	$O(V+E)$	$O(V+E)$	$O(V)$
Greedy Ranking	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
0/1 Knapsack DP	$O(nW)$	$O(nW)$	$O(nW)$	$O(nW)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Boyer–Moore–Horspool	$O(n/m)$	sublinear	$O(n \cdot m)$	$O(\sigma)$
Branch and Bound	$O(n^2)$	$\ll O(n!)$	$O(n!)$	$O(n^2)$

5.9 Advantages

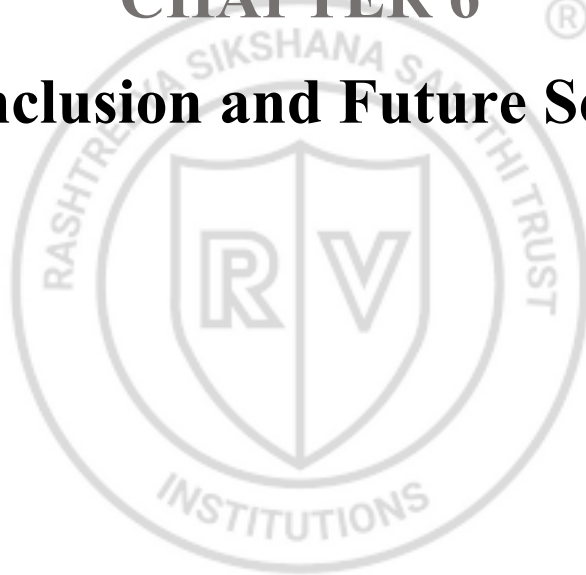
- **Algorithmically grounded decisions.** Planning choices are computed by provably correct algorithms rather than entered manually, reducing blocked work and under-utilized sprints.
- **Optimal sprint composition.** The 0/1 Knapsack solver maximizes delivered business value within the capacity constraint, outperforming the greedy heuristic in non-trivial cases.
- **Skill-aware assignment.** Branch and Bound produces an optimal member allocation that minimizes total skill mismatch across the whole team simultaneously.
- **Predictable performance.** Merge Sort's guaranteed $O(n \log n)$ behavior and the linear graph algorithms keep the interface responsive.
- **Cycle safety.** Invalid dependency configurations are detected and reported before optimization, preventing inconsistent plans.
- **Modular design.** Each algorithm is an independent service, making the system easy to test, extend, and reason about.

5.10 Limitations

- **Pseudo-polynomial knapsack.** The knapsack solver's cost depends on the numeric capacity W , so very large capacities increase its running time.
- **Exponential worst case for assignment.** Branch and Bound can degrade toward $O(n!)$ on adversarial inputs; very large teams would require approximate assignment methods.
- **Single-resource capacity model.** Sprint capacity is modeled as one scalar; multiple simultaneous resource constraints are not yet supported.
- **Dependence on input quality.** Optimization quality depends on the accuracy of the effort, value, and skill estimates supplied to the algorithms.
- **Static skill model.** Member skills are treated as fixed during assignment and do not yet account for learning or availability over time.

CHAPTER 6 [®]

Conclusion and Future Scope



6.1 Conclusion

This project designed, analyzed, and implemented NexusFlow, an AI-powered project management and sprint optimization system in which classical algorithms drive the core planning decisions. By modeling tasks and dependencies as a directed graph, the system uses Breadth First Search to identify execution-ready work and Topological Sort to compute a valid execution order while detecting invalid cycles. Greedy ranking and 0/1 Knapsack Dynamic Programming together provide both a fast preview and an exact, value-maximizing sprint backlog under a capacity constraint. Merge Sort delivers stable, predictable multi-criteria ordering, Boyer–Moore–Horspool enables fast task search, and Branch and Bound computes an optimal, skill-aware assignment of members to tasks.

Each algorithm was analyzed for correctness and asymptotic time and space complexity, and the observed behavior matched the theoretical predictions. The project therefore demonstrates that the techniques studied in the Design and Analysis of Algorithms course are directly applicable to a substantial, realistic engineering problem, and that combining them within a single coherent workflow yields a planning system that reduces blocked work, balances workload against skill, and maximizes delivered value per sprint.

6.2 Learning Outcomes of the Project

- Gained the ability to recognize standard algorithmic problems—graph ordering, bounded subset selection, and assignment—within a real application domain.
- Developed proficiency in deriving and verifying time and space complexity using asymptotic analysis across multiple algorithm paradigms.
- Implemented divide-and-conquer, greedy, dynamic-programming, and branch-and-bound algorithms and integrated them into a working full-stack system.
- Learned to distinguish heuristic and exact methods and to justify when each is appropriate based on input size and optimality requirements.
- Acquired practical experience building a React Native, Node.js, and MongoDB application and exposing algorithmic services through a REST API.

6.3 Future Scope

- Extend the capacity model to handle multiple simultaneous resource constraints using multidimensional knapsack formulations.
- Incorporate the Hungarian algorithm as a polynomial-time alternative to Branch and Bound for very large assignment instances.
- Add velocity-based forecasting so that sprint capacity adapts automatically from historical team performance.
- Introduce time-aware scheduling that accounts for member availability and task duration, moving beyond a single scalar capacity.
- Enhance the AI planner with feedback loops that refine effort and value estimates from completed sprints.

6.4 References / Bibliography

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA: MIT Press, 2022.
2. A. Levitin, Introduction to the Design and Analysis of Algorithms, 3rd ed. Boston, MA: Pearson, 2012.
3. J. Kleinberg and É. Tardos, Algorithm Design. Boston, MA: Pearson/Addison-Wesley, 2006.
4. S. Dasgupta, C. H. Papadimitriou, and U. V. Vazirani, Algorithms. New York, NY: McGraw-Hill, 2008.
5. R. N. Horspool, “Practical fast searching in strings,” Software: Practice and Experience, vol. 10, no. 6, pp. 501–506, 1980.
6. R. S. Boyer and J. S. Moore, “A fast string searching algorithm,” Communications of the ACM, vol. 20, no. 10, pp. 762–772, 1977.
7. A. H. Land and A. G. Doig, “An automatic method of solving discrete programming problems,” Econometrica, vol. 28, no. 3, pp. 497–520, 1960.
8. H. W. Kuhn, “The Hungarian method for the assignment problem,” Naval Research Logistics Quarterly, vol. 2, no. 1–2, pp. 83–97, 1955.
9. K. Schwaber and J. Sutherland, The Scrum Guide, Scrum.org, 2020.
10. React Native Documentation, Meta Platforms, Inc. [Online]. Available: <https://reactnative.dev>
11. Node.js Documentation, OpenJS Foundation. [Online]. Available: <https://nodejs.org/en/docs>
12. MongoDB Manual, MongoDB, Inc. [Online]. Available: <https://www.mongodb.com/docs>